

Prompt Completo

01 Descripcion General

Crea un programa en Python llamado **inicio.py** para un taller academico de programacion estructurada. El programa debe usar unicamente funciones definidas con `def`, listas, diccionarios y estructuras de control basicas (`for`, `while`, `if`). **No se permite** programacion orientada a objetos ni funciones `lambda`.

02 Modulo de Autenticacion

■ registrar_usuario()

RECIBE

- Nada — pide datos por consola

HACE

- Solicita correo y contrasena con `input()`
- Crea un diccionario con ambos datos
- Agrega el dict a lista global con `.append()`
- Imprime confirmacion en pantalla

ENTREGA

- Nada (None)

input() Lee texto escrito por el usuario en consola

Diccionario {} Empaqueta correo y password en un objeto

.append(nuevo) Agrega el dict al final de la lista global

```
1 usuarios_registrados = [] # lista global

3 def registrar_usuario():
4     correo = input("Correo: ").strip()
5     password = input("Contrasena: ").strip()

7     # Empaquetar datos en un diccionario
8     nuevo = {
9         "correo": correo,
10        "password": password
11    }

13    # append agrega al final de la lista
14    usuarios_registrados.append(nuevo)
15    print("Usuario registrado exitosamente")
```

■ iniciar_sesion()

RECIBE

- Nada — lee lista global

HACE

- Repite el proceso hasta 3 veces con while
- Busca coincidencia con for + if
- Avisa cuantos intentos quedan si falla
- Bloquea si se agotan los 3 intentos

ENTREGA

- True (OK) o False (bloqueado)

while + contador Controla que no se superen 3 intentos

for + if Recorre la lista buscando coincidencia exacta

return True/False Comunica el resultado a main()

```

1 def iniciar_sesion():
2     MAX = 3
3     intentos = 0

4
5     while intentos < MAX:
6         correo = input("Correo: ")
7         pwd = input("Contraseña: ")

8
9         for u in usuarios_registrados:
10            if u["correo"]==correo and u["password"]==pwd:
11                print("Login exitoso")
12                return True

13
14            intentos += 1
15            print(f"Intentos restantes: {MAX - intentos}")

16
17    print("Cuenta bloqueada temporalmente")
18    return False

```

03 Modulo de Datos



crear_usuarios()

RECIBE

- Nada — construye datos internamente

HACE

- Construye lista con 10 diccionarios
- Campos: id, nombre, documento, estrato
- consumos: lista de 30 valores KWH
- estado: 'ACTIVO' o 'SUSPENDIDO'

ENTREGA

- lista de 10 diccionarios

Lista []

Contiene los 10 diccionarios como elementos

Diccionario { }

Cada cliente es un dict con 6 campos

Lista interna

consumos es una lista de 30 numeros dentro del dict

return usuarios

Devuelve la lista para que main() la use

```
1 def crear_usuarios():
2     # Lista con 10 diccionarios
3     usuarios = [
4         {
5             "id": 1,
6             "nombre": "Carlos Ramirez",
7             "documento": "1020304050",
8             "estrato": 3,
9             # lista de 30 consumos en KWH
10            "consumos": [120, 95, 110, 88, 134, ...],
11            "estado": "ACTIVO"
12        },
13        # ... 9 diccionarios mas
14    ]
15    return usuarios
```

■ mostrar_usuarios(usuarios)

RECIBE

- usuarios — lista de 10 diccionarios

HACE

- Recorre cada diccionario con for
- Suma los 30 consumos con sum()
- Imprime: ID, nombre, estrato, total KWH

ENTREGA

- Nada (None) — imprime en pantalla

for u in usuarios Itera uno a uno los 10 diccionarios

sum(lista) Suma los 30 valores de consumos automaticamente

print(f"...") f-string para mostrar datos formateados

```
1 def mostrar_usuarios(usuarios):
2     print("ID    Nombre           Estrato   Total KWH")
3     print("-" * 45)

4
5     # for recorre cada uno de los 10 diccionarios
6     for u in usuarios:
7         # sum() suma los 30 valores de la lista consumos
8         total = sum(u["consumos"])

9
10        print(
11            f"{u['id']:<4}"
12            f"{u['nombre']:<18}"
13            f"{u['estrato']:<9}"
14            f"{total} KWH"
15        )
```

■ ordenar_por_consumo(usuarios)

RECIBE

- usuarios — lista de 10 diccionarios

HACE

- Recorre con dos for anidados (burbuja)
- Compara consumo total de vecinos
- Intercambia si el orden es incorrecto
- Repite hasta ordenar toda la lista

ENTREGA

- usuarios — lista ordenada menor a mayor

for i + for j Doble bucle = algoritmo de burbuja (Bubble Sort)

sum(consumos) Calcula el consumo total de cada usuario

a, b = b, a Python intercambia valores en una sola linea

return usuarios Devuelve la lista ya ordenada

```
1 def ordenar_por_consumo(usuarios):
2     n = len(usuarios)

4     # Algoritmo de burbuja (Bubble Sort)
5     for i in range(n):
6         for j in range(n - i - 1):
7             c1 = sum(usuarios[j]["consumos"])
8             c2 = sum(usuarios[j+1]["consumos"])

10            if c1 > c2:
11                # intercambiar posiciones
12                usuarios[j], usuarios[j+1] = \
13                    usuarios[j+1], usuarios[j]

15     return usuarios
```

04 Modulo de Interfaz



menu_principal(usuarios)

RECIBE

- usuarios — lista de 10 diccionarios

HACE

- Muestra el menu con while True
- Opcion 1 -> mostrar_usuarios()
- Opcion 2 -> ordenar y mostrar
- Opcion 3 -> break, termina

ENTREGA

- Nada (None)

while True Ciclo infinito: repite hasta que el usuario salga

input() Captura la opcion ingresada por el usuario

if / elif Cada opcion lleva a una accion diferente

break Rompe el while y finaliza limpiamente

```
1 def menu_principal(usuarios):
2     while True:
3         print("\n1. Mostrar usuarios")
4         print("2. Ordenar por consumo")
5         print("3. Salir")
6         op = input("Opcion: ")
7
8         if op == "1":
9             mostrar_usuarios(usuarios)
10
11        elif op == "2":
12            usuarios = ordenar_por_consumo(usuarios)
13            mostrar_usuarios(usuarios)
14
15        elif op == "3":
16            print("Hasta luego.")
17            break
```

05 Funcion Principal

■ main()

RECIBE

- Nada — punto de entrada del programa

HACE

- Llama a registrar_usuario()
- Llama a iniciar_sesion() -> guarda en acceso
- Si True -> crear_usuarios() + menu_principal()
- Si False -> mensaje y termina

ENTREGA

- Nada (None) — dirige el flujo

Secuencia

Ejecuta funciones en orden, una tras otra

if / else

Decide el camino segun el resultado del login

```

1 def main():
2     # 1. Registrar usuario
3     registrar_usuario()

5     # 2. Iniciar sesion (max 3 intentos)
6     acceso = iniciar_sesion()

8     if acceso:
9         # 3. Cargar datos del servicio
10        usuarios = crear_usuarios()
11        # 4. Mostrar menu interactivo
12        menu_principal(usuarios)

14    else:
15        print("Programa finalizado.")

17 # Punto de entrada del programa
18 if __name__ == "__main__":
19     main()

```

06 Resumen de Contratos por Funcion

Funcion	Recibe	Hace	Entrega
main()	Nada	Orquesta todo el flujo	Nada
registrar_usuario()	Nada	Guarda usuario en lista global	Nada
iniciar_sesion()	Nada	Verifica credenciales (3 int.)	True / False
crear_usuarios()	Nada	Construye 10 diccionarios	Lista de 10 dicts
mostrar_usuarios()	Lista usuarios	Imprime tabla con consumo total	Nada
ordenar_por_consumo()	Lista usuarios	Ordena por consumo (burbuja)	Lista ordenada
menu_principal()	Lista usuarios	Gestiona menu interactivo	Nada

inicio.py · Python Estructurado · def · for · while · if · list · dict